Lập trình mạng · Chương 2

# Bài toán ứng dụng: dịch vụ tra cứu qua TCP

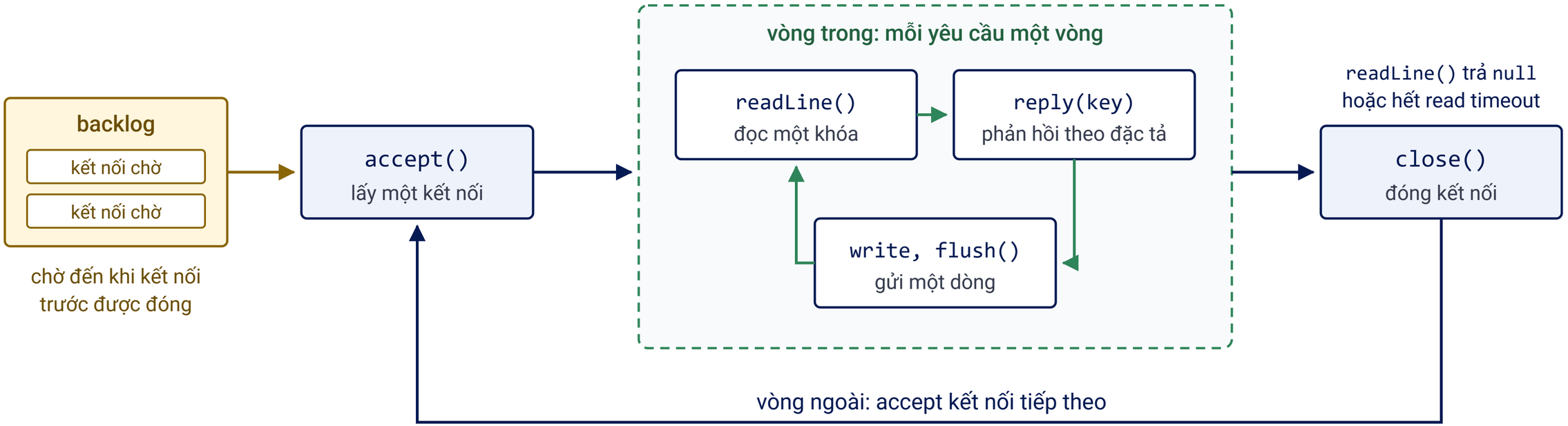
TS. Đặng Ngọc Hùng

# Dịch vụ tra cứu ghép đặc tả, kết nối và timeout vào một server thật

1. Đã có: **đặc tả** giao thức tra cứu, kết nối TCP, read timeout, half-close
2. Cần: trả lời **nhiều yêu cầu** trong một kết nối, phục vụ **nhiều client**, trả đúng mã lỗi
3. Và cần **kiểm thử tự động**: chạy lại các trường hợp thử của đặc tả sau mỗi lần sửa mã

> **Câu hỏi của phần:** ghép các cơ chế đã học thành dịch vụ tra cứu như thế nào, và kiểm thử dịch vụ đó ra sao?

# Server tuần tự phục vụ xong một kết nối rồi mới accept kết nối sau




**Server tuần tự**: vòng ngoài accept từng kết nối, vòng trong trả lời từng yêu cầu; kết nối sau chờ trong backlog.

# Mọi kết nối cùng local port 9199, phân biệt bằng 4-tuple

**listening socket**
`port 9199`
không trao đổi dữ liệu

| kết nối | địa chỉ client | port client | địa chỉ server | port server |
|---|---|---|---|---|
| #1 | 127.0.0.1 | 53094 | 127.0.0.1 | 9199 |
| #2 | 127.0.0.1 | 53095 | 127.0.0.1 | 9199 |
| #3 | 127.0.0.1 | 53096 | 127.0.0.1 | 9199 |

**khác nhau: ephemeral port**
hệ điều hành phía client cấp

ba cột còn lại
giống nhau

chỉ cần một trong bốn giá trị khác nhau là **hai kết nối khác nhau**

**4-tuple** (địa chỉ và port của hai đầu) định danh một kết nối TCP. Server không cấp port mới cho mỗi client.

# Vòng lặp ngoài: một kết nối tại một thời điểm, lỗi của một kết nối không dừng server

Java

```
void serve() {
    while (!listener.isClosed()) {
        try (Socket connected = listener.accept()) {  // một kết nối tại một thời điểm
            connected.setSoTimeout(5_000);  // read timeout: client không gửi gì quá 5 giây
            log.println("accepted " + fourTuple(connected));
            int requests = handle(connected);  // phục vụ đến khi client đóng chiều gửi
            log.println("closed, requests: " + requests);
        } catch (IOException e) {  // gồm cả SocketTimeoutException: lỗi của một kết nối không dừng server
            log.println("connection error: " + e.getMessage());
        }
    }
}
```

Kết quả chạy

```
listening on port 9199
accepted local 127.0.0.1:9199, remote 127.0.0.1:53094
closed, requests: 5
accepted local 127.0.0.1:9199, remote 127.0.0.1:53095
closed, requests: 1
accepted local 127.0.0.1:9199, remote 127.0.0.1:53096
closed, requests: 1
```

# Vòng lặp trong: mỗi dòng nhận được là một khóa, trả đúng một dòng

Java

```
int handle(Socket connected) throws IOException {
    var in = new BufferedReader(new InputStreamReader(connected.getInputStream(), UTF_8));
    var out = new BufferedWriter(new OutputStreamWriter(connected.getOutputStream(), UTF_8));
    int requests = 0;
    for (String key; (key = in.readLine()) != null; requests++) {  // null: client đã đóng chiều gửi
        out.write(reply(key) + "\n");  // OK <giá trị>, NOTFOUND hoặc ERR <lý do>
        out.flush();
    }
    return requests;
}
```

Kết quả chạy

```
hanoi -> OK Thủ đô của Việt Nam (17 ms)
huế -> OK Cố đô triều Nguyễn (0 ms)
atlantis -> NOTFOUND (0 ms)
(empty line) -> ERR empty key (0 ms)
xxxxxxxxxxxxxxxxxxxx… (300 bytes) -> ERR line too long (8 ms)
```

# Integration test chạy server thật ở port 0 và kiểm các trường hợp của đặc tả

Java

```java
@BeforeAll
static void startServer() throws IOException {
    listener = new ServerSocket(0, 50, InetAddress.getLoopbackAddress());  // port 0: hệ điều hành cấp port trống
    Thread server = new Thread(() -> new LookupServer(listener, NO_LOG).serve());
    server.setDaemon(true);
    server.start();  // server chạy song song với test; thread học ở nhóm đa luồng
}

@Test
void knownKey() throws IOException {
    assertEquals("OK Thủ đô của Việt Nam", ask("hanoi"));
}
```

Kết quả chạy

```
└─ JUnit Jupiter ✓
   └─ LookupServerTest ✓
      ├─ portZeroGetsEphemeralPort() ✓
      ├─ knownKey() ✓
      ├─ unknownKey() ✓
      ├─ emptyLine() ✓
      ├─ lineLongerThan256Bytes() ✓
      └─ crlfIsAccepted() ✓

Test run finished after 175 ms
```

# Một client không gửi gì làm mọi client sau phải chờ

Java

```
connected.setSoTimeout(5_000);  // read timeout: client không gửi gì quá 5 giây
int requests = handle(connected);  // phục vụ đến khi client đóng chiều gửi
```

Kết quả chạy

```
listening on port 9199
accepted local 127.0.0.1:9199, remote 127.0.0.1:53098
connection error: Read timed out
accepted local 127.0.0.1:9199, remote 127.0.0.1:53099
closed, requests: 1
```

- Client tra cứu `hanoi` đến sau **1 giây**: nhận phản hồi sau **3983 ms**, thay vì vài mili giây
- Server tuần tự bị chặn ở `readLine()` của kết nối đầu; kết nối sau đã bắt tay xong, **nằm chờ trong backlog**

# Câu hỏi 1

**Ba client trên cùng một máy kết nối lần lượt tới server tra cứu ở port 9199. Ba kết nối được phân biệt bằng gì?**

A. Bằng 4-tuple: địa chỉ và port của hai đầu; ở đây ba kết nối khác nhau ở port client.

B. Server cấp cho mỗi kết nối một port mới sau khi accept.

C. Bằng thứ tự gọi accept: kết nối thứ nhất, thứ hai, thứ ba.

D. Bằng địa chỉ IP của client.

# Câu hỏi 1

## Ba client trên cùng một máy kết nối lần lượt tới server tra cứu ở port 9199. Ba kết nối được phân biệt bằng gì?

A. Bằng 4-tuple: địa chỉ và port của hai đầu; ở đây ba kết nối khác nhau ở port client.

B. Server cấp cho mỗi kết nối một port mới sau khi accept.

C. Bằng thứ tự gọi accept: kết nối thứ nhất, thứ hai, thứ ba.

D. Bằng địa chỉ IP của client.

Một kết nối TCP được định danh bằng 4-tuple. Cả ba kết nối có local port 9199 và cùng địa chỉ 127.0.0.1 ở hai đầu; chỉ port client (ephemeral port) khác nhau. Server không cấp port mới cho kết nối.

# Câu hỏi 2

**Một client kết nối tới server tra cứu tuần tự nhưng không gửi dòng nào. Vì sao các client đến sau phải chờ?**

A. Backlog chỉ chứa được một kết nối.

B. Kết nối của các client sau bị từ chối (connection refused) cho đến khi client đầu đóng.

C. TCP chỉ cho phép một kết nối tới mỗi port tại một thời điểm.

D. Server đang bị chặn ở `readLine()` của kết nối đầu, nên chưa gọi `accept()` cho các kết nối sau.

# Câu hỏi 2

## Một client kết nối tới server tra cứu tuần tự nhưng không gửi dòng nào. Vì sao các client đến sau phải chờ?

A. Backlog chỉ chứa được một kết nối.

B. Kết nối của các client sau bị từ chối (connection refused) cho đến khi client đầu đóng.

C. TCP chỉ cho phép một kết nối tới mỗi port tại một thời điểm.

**D. Server đang bị chặn ở `readLine()` của kết nối đầu, nên chưa gọi `accept()` cho các kết nối sau.**

Kết nối của client sau đã bắt tay xong và nằm trong backlog; chỉ có server chưa gọi `accept()` vì còn chờ ở `readLine()` của kết nối đầu, đến khi read timeout hết hạn. TCP cho phép nhiều kết nối tới cùng một port.

# Tổng kết

1. **Server tuần tự**: vòng ngoài accept từng kết nối, vòng trong trả lời từng yêu cầu đến khi `readLine()` trả `null`

2. Mọi kết nối cùng port 9199, phân biệt bằng **4-tuple**; port client là **ephemeral port**

3. **Integration test** chạy server thật ở **port 0**, kiểm các trường hợp thử của đặc tả

**Từ khóa cần nhớ** server tuần tự · 4-tuple · integration test · port 0 · ephemeral port

# Mở rộng

### Chủ đề nâng cao

- Đọc dòng có giới hạn: dừng ở byte thứ 257 thay vì giữ cả dòng trong bộ nhớ
- Integration test cho lỗi mạng: client đóng giữa chừng, kết nối bị reset

### Bài tập mở rộng

Hiện thực lệnh STATS theo đặc tả đã bổ sung ở Chương 1; viết integration test cho lệnh mới.

Tiếp theo

# Đa luồng: thread và race condition